

Web API Services Fundamentals

Day 4



Agenda for Day 4

- Introduction to Web API
- Why use APIs?
- RESTful Principles (Review)
- Web API vs. MVC
- Web API Project Setup & Basics
- Controllers, Actions, Routing, Request/Response Handling
- Consuming Web APIs from an MVC Application (**HttpClient** , **HttpClientFactory**)
- Lab Assignment

What is a Web API?

- **API (Application Programming Interface):** A set of defined rules that enable different software applications to communicate with each other.
- **Web API:** An API that uses HTTP as its communication protocol. (use HTTP method like **GET, POST, PUT, DELETE**)
- Allows diverse clients (web browsers, mobile apps, desktop apps, other services) to interact with your backend data and logic.
- Typically returns data in formats like JSON or XML.

Example

```
Client (e.g., Web Browser, Mobile App) <---HTTP Request---> Web API Server
Client sends: GET /api/products
Web API Server processes request, fetches data from database, and sends back:
HTTP/1.1 200 OK
Content-Type: application/json
[
  { "id": 1, "name": "Laptop", "price": 1200.00 },
  { "id": 2, "name": "Mouse", "price": 25.00 }
]
```

Why Use APIs?

- **Decoupling:** Separates frontend (client) from backend (server).
- **Reusability:** A single backend can serve multiple types of clients.
- **Scalability:** Easier to scale backend services independently.
- **Flexibility:** Allows different technologies for frontend and backend.
- **Integration:** Enables third-party integrations.

RESTful Principles (Review)

- **Representational State Transfer (REST):** An architectural style for networked applications.
- **Key Principles:**
 - **Stateless:** Each request from client to server must contain all information needed to understand the request.
 - **Client-Server:** Separation of concerns.
 - **Uniform Interface:** Consistent way of interacting with resources (e.g., standard HTTP methods).
 - **Cacheable:** Responses can be cached.
 - **Layered System:** Intermediaries can be used.

Web API vs. MVC

■ ASP.NET Core MVC:

- **Purpose:** Primarily for building web applications that render HTML views to browsers.
- **Output:** HTML, CSS, JavaScript.
- **Interaction:** User interacts directly with the rendered web page.

■ ASP.NET Core Web API:

- **Purpose:** Primarily for building HTTP services that expose data and operations.
- **Output:** Data formats like JSON, XML.
- **Interaction:** Consumed by other applications (SPAs, mobile apps, other services).

Web API Project Setup

- **Visual Studio:** ASP.NET Core Web API" template.
- **.NET CLI:** `dotnet new webapi -n MyWebApi`.
- **Key Differences from MVC Project:**
 - No **Views** folder by default.
 - `Program.cs` typically calls `builder.Services.AddControllers()` (not `AddControllersWithViews()`).
 - Routing is primarily **attribute-based**.

Web API Project Setup

■ Web API Documentation

- `builder.Services.AddOpenApi();` used for adding OpenAPI service for documentation of the API

```
builder.Services.AddOpenApi();  
...  
app.MapOpenApi(); //Register an endpoint /openapi/v1.json
```

- Add `Swashbuckle.AspNetCore` nugetPackage for UI documentation

```
builder.Services.AddSwaggerGen();  
app.UseSwagger();  
app.UseSwaggerUI(c =>  
{  
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "WebAPI1 v1");  
    //c.RoutePrefix = string.Empty; // Set Swagger UI at the app's root  
});
```

Web API Controllers & Actions

- **Controllers:**
- Inherit from **ControllerBase** (lighter than **Controller**, no view-related features).
- Decorated with **[ApiController]** attribute:
 - Enables automatic model validation.
 - Automatic HTTP 400 responses for validation errors.
 - Infers parameter binding sources.
- **Actions:** Public methods that respond to HTTP verbs.
 - **[HttpGet]**, **[HttpPost]**, **[HttpPut]**, **[HttpDelete]**, **[HttpPatch]**.

Web API Routing

- **Attribute Routing:** The standard and recommended approach for Web APIs.
- `[Route("api/[controller]")]` on the controller.
- `[HttpGet]`, `[HttpGet("{id}")]`, `[HttpPost]` on actions.
- Provides clear, explicit, and flexible routing.

Web API Request & Response Handling

■ **Model Binding:**

- **[FromBody]**: Data from the request body (e.g., JSON payload for POST/PUT).
- **[FromQuery]**: Data from the query string.
- **[FromRoute]**: Data from the URL route.

■ **Automatic Validation:** `[ApiController]` handles `ModelState.IsValid` check automatically.

Web API Request & Response Handling

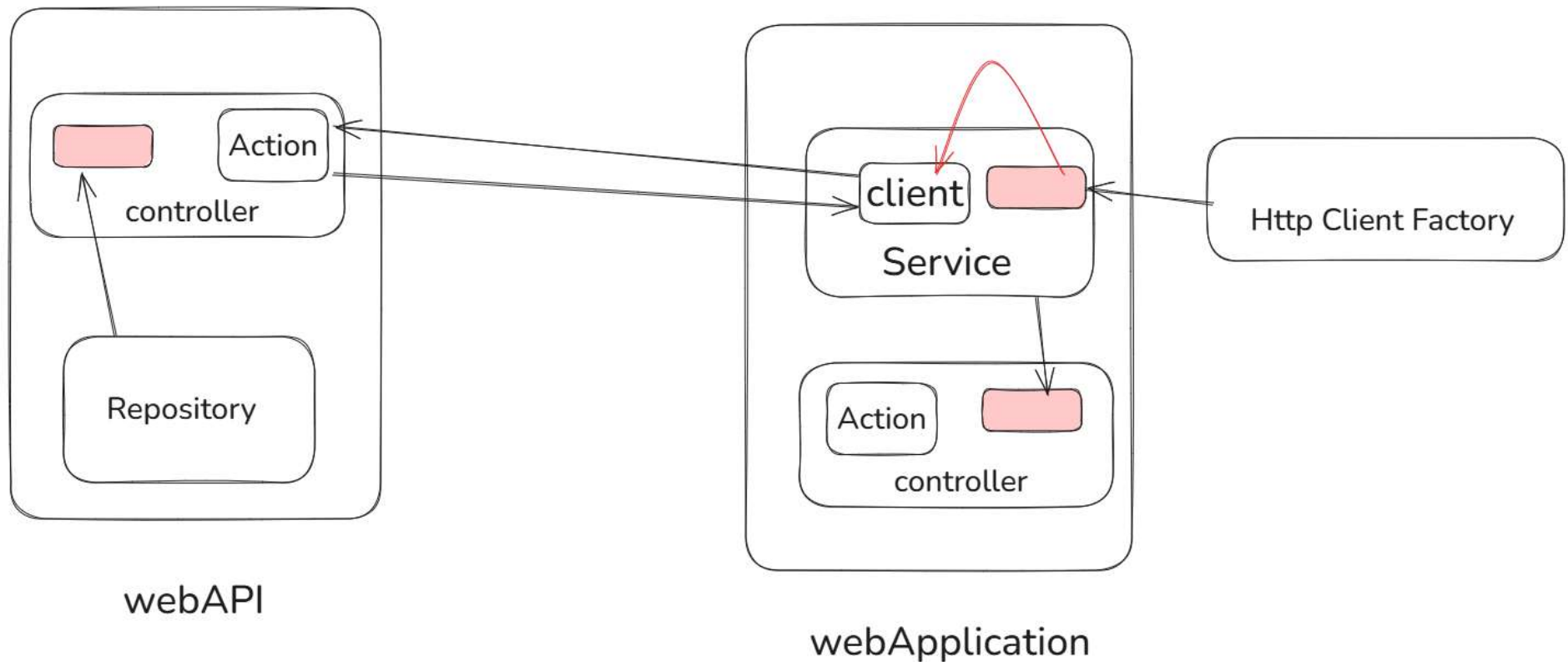
■ Action Results:

- `Ok(data)` (200 OK)
- `NotFound()` (404 Not Found)
- `BadRequest(ModelState)`` (400 Bad Request)
- `CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product)` (201 Created)
- `NoContent()` (204 No Content)

Consuming Web APIs from an MVC Application

- **HttpClient**: The primary class for sending HTTP requests.
- **HttpClientFactory`**:
 - **Purpose**: Manages **HttpClient** instances to avoid common issues like socket exhaustion.
 - **Registration**: `builder.Services.AddHttpClient("ClientName", ...)` in `Program.cs`.
 - **Usage**: Inject **IHttpClientFactory** and create named clients.
- **Serialization/Deserialization:****
 - `System.Net.Http.Json` extension methods (used by **HttpClient**)
 - (`GetFromJsonAsync` , `PostAsJsonAsync` , `PutAsJsonAsync`).
 - `Newtonsoft.Json` (if preferred).

Consuming Web APIs from an MVC Application



Consuming API Example (MVC Service)

```
public class ProductApiService
{
    private readonly HttpClient _httpClient;
    public ProductApiService(HttpClient httpClient) { _httpClient = httpClient; }

    public async Task<List<Product>> GetProductsAsync()
    {
        return await _httpClient.GetFromJsonAsync<List<Product>>("api/Products");
    }

    public async Task<Product> CreateProductAsync(Product product)
    {
        var response = await _httpClient.PostAsJsonAsync("api/Products", product);
        response.EnsureSuccessStatusCode(); // Throws on error status
        return await response.Content.ReadFromJsonAsync<Product>();
    }
}
```


API Versioning

- **Why?** To evolve APIs without breaking existing clients.
- **Strategies:**
 - **URL Versioning:** `api/v1/products` (Clear, but URL pollution).
 - **Query String Versioning:** `api/products?api-version=1.0` (Flexible).
 - **Header Versioning:** Custom HTTP header (Clean).
- Implemented using `Microsoft.AspNetCore.Mvc.Versioning` (`Asp.Versioning.Mvc` → new package)package.

API Versioning

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    // You can add different versioning schemes here, e.g.,
    // options.ApiVersionReader = new UrlSegmentApiVersionReader();
    // options.ApiVersionReader = new QueryStringApiVersionReader("api-version");
    // options.ApiVersionReader = new HeaderApiVersionReader("X-API-Version");
});
```

API Versioning

Assignment

- Create WebAPI that preform CURD on Movie model
 - Enable swagger UI documentation
 - **Optinal**: using **DbContext** (database)
- Create Web MVC application that consume WebAPI created previously to Preform CURD on movie

